

Machine Learning su Dataset con Deployment tramite Flask e Docker

Brivio Emanuele

Combi Federico



OBIETTIVO DEL PROGETTO

- Caricare un dataset CSV
- Pulire e analizzare i dati
- Addestrare vari modelli di regressione:
 - Linear
 - Lasso
 - Ridge
- Creare l'interfaccia con libreria Flask
- Containerizzazione con Docker

DATASET

Il dataset è stato prelevato da *UC Irvine Machine Learning Repository* nello specifico *Auto MPG*

Contiene variabili numeriche continue e discrete

Variabile target: mpg (*miles per gallon* cioè *miglia per gallone*)

Problemi iniziali:

- Valori mancanti: nella colonna *horsepower*
- Possibili outlier

Variable Name	Role	Type	Description	Units	Missing Values
displacement	Feature	Continuous			no
mpg	Target	Continuous			no
cylinders	Feature	Integer			no
horsepower	Feature	Continuous			yes
weight	Feature	Continuous			no
acceleration	Feature	Continuous			no
model_year	Feature	Integer			no
origin	Feature	Integer			no
car_name	ID	Categorical			no

Struttura CSV

DATA CLEANING

```
def clean_data(self, lista : {__iter__}): 3+ usages  @ Federico Combi *  
    # togliamo le colonne non importanti per il calcolo le droppiamo a priori  
    # queste sono specificate dall'utente  
    self.data.drop_duplicates()  
  
    # tolgo valori NaN da horsepower  
    # se dati sono pochi rispetto tot (<5%) posso dropare la riga direttamente  
    self.data["horsepower"] = self.data["horsepower"].fillna(  
        self.data["horsepower"].mean() # meglio fare mediana -> controlla  
    )  
  
    for colonna in lista:  
        self.data = self.data.drop(colonna, axis=1)  
  
    threshold = 1  
    cols_to_keep = [  
        col for col in self.data.columns  
        if self.data[col].value_counts(normalize=True, dropna=False).max() <= threshold  
    ]  
    self.data = self.data[cols_to_keep]
```

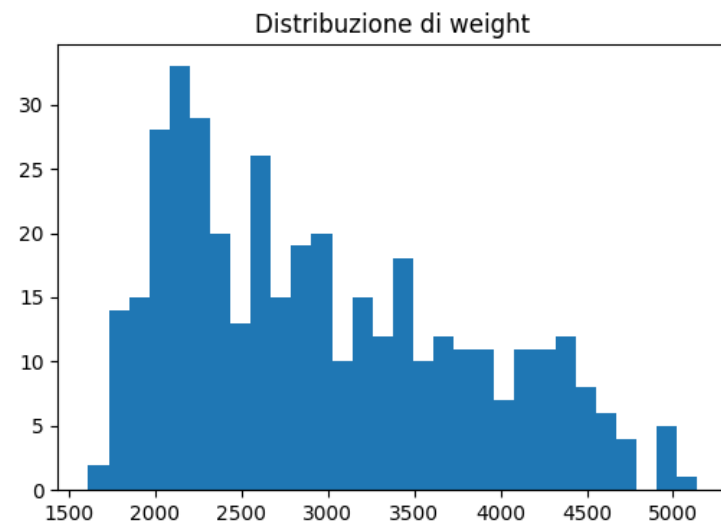
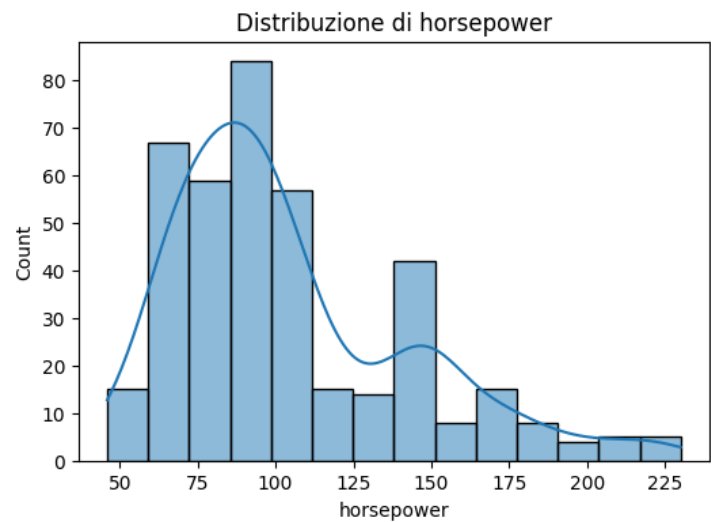
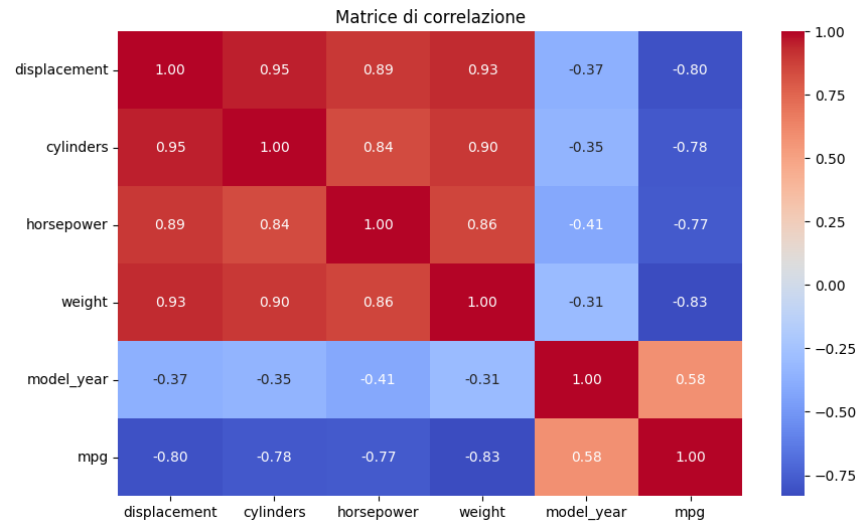
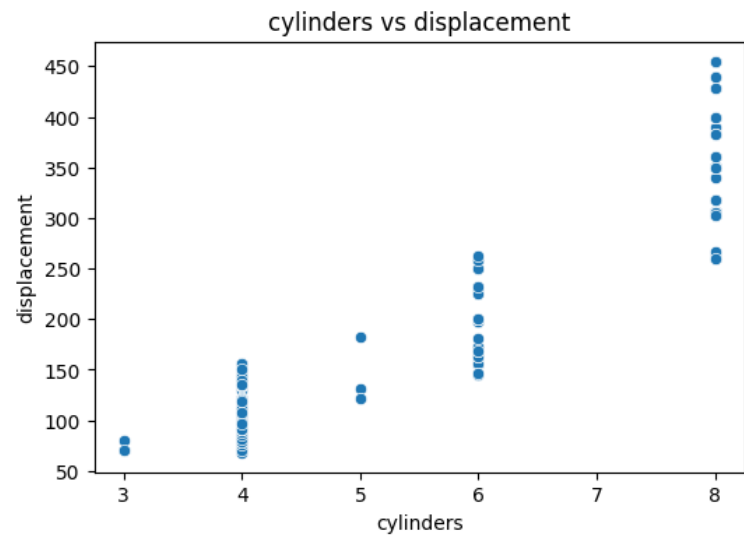
Operazioni effettuate:

- Rimozione duplicati
- Gestione dei valori *NaN* di *horsepower*
- Drop delle colonne non strettamente necessarie
- Drop colonne con valore invariato per ogni entrata

EXPLORATION DATA ANALYSIS (EDA)

- Analisi univariata:
 - Summary delle varie variabili numeriche
 - Valutazione della distribuzione, tramite hist plot
 - Individuazione outlier tramite Z score, IQR ovvero quanto un valore si discosta dalla media e quali valori che stanno fuori dal 25-75 percentile
- Analisi multivariata:
 - Matrice di correlazione
 - Scatter plot

```
@self.app.route("/info_csv")  ⚡ Federico Combi
def info_csv():
    dt = {
        "head": self.ds_mn.mostra_cont(5).to_json(),
        "shape and info": self.ds_mn.struttura(),
        "summary num": self.ds_mn.summary().to_json(),
        "missing values": self.ds_mn.missing_values().to_json(),
        "missing v in %": self.ds_mn.missing_percentage().to_json(),
        "mat correlazione": self.ds_mn.correlation_matrix().to_json()
    }
    return jsonify(dt)
```



MODELLI MACHINE LERNING SPIEGAZIONE DEI RISULTATI

```
"predizioni": y_pred.tolist(),  
"coeff": self.linear.coef_.tolist(),  
"intercetta": float(self.linear.intercept_),  
"MSE": float(mse),  
"R2": float(r2)
```

Spiegazione risultati:

- Predizioni: valori predetti dal nostro algoritmo sul test set
- Coeff: parametri attribuiti alle variabili esplicative
- Intercetta: punto d'origine della nostra retta di previsione
- MSE: metrica per confrontare l'errore nei diversi modelli
- R^2 : indica quanto il modello riesce a rappresentare la struttura dati

MODELLO MACHINE LEARNING

RISULTATI

200 OK

Headers >

```
{
  "MSE": 9.484329876353195,
  "R2": 0.8236013168277808,
  "coeff": [
    0.000903361031726556,
    0.05577285861771808,
    -0.003976405882964225,
    -0.006788058884491362,
    0.7976565803380773
  ],
  "intercetta": -17.008122458476755,
  "predizioni": [
    31.523759726116012,
    30.59842125097964,
    21.892323966546932,
    17.00581643690731,
    12.596793207199141,
    26.129021405280447,
    28.93566106285771,
    9.068083546200413,
    17.095567246584856,
    22.602691863620862,
    12.736788013448809,
    35.10005951541944,
    26.38178850289661,
    14.128369887566414,
    24.814550339854193,
    6.067080058031486,
```

REGRESSIONE LINEARE

200 OK

Headers >

```
{
  "MSE": 9.238900430777477,
  "R2": 0.8281660495580485,
  "coeff": [
    0,
    0,
    -0.08582325929873592,
    -5.505649783511167,
    2.7849592142068165
  ],
  "intercetta": 23.60817610062894,
  "predizioni": [
    31.345866606557788,
    30.485708365528637,
    21.862475690806548,
    17.050034470767322,
    12.779351566803014,
    26.16740090156642,
    28.88119594748825,
    9.361804781092244,
    17.1297518274879,
    22.593593904825592,
    12.902842924517035,
    34.84303905204757,
    26.386187220977227,
    14.268364381142652,
    24.84636649781632,
    6.4515343875707885,
```

REGRESSIONE LASSO

200 OK

Headers >

```
{
  "MSE": 9.455731532830399,
  "R2": 0.8241332163087243,
  "coeff": [
    -0.005909338546716488,
    0.07414729750552826,
    -0.17160348632270545,
    -5.555900538116658,
    2.849186524085175
  ],
  "intercetta": 23.60817610062894,
  "predizioni": [
    31.473639808801057,
    30.608974135859015,
    21.82185329245615,
    16.98779592448788,
    12.645771025456726,
    26.124027455243983,
    28.958809770218483,
    9.040730675826095,
    17.174232423643424,
    22.583581417685473,
    12.642018056791283,
    35.035302568210646,
    26.375709595937987,
    14.138391203688373,
    24.839897743211356,
    6.165985204329026,
    30.91259793997407,
    23.904236274742953,
```

REGRESSIONE RIDGE

MACHINE LEARNING

INTERPRETAZIONE DEI RISULTATI

REGRESSIONE LINEARE

- Il valore di R^2 è **molto elevato**, segno di un'ottima capacità del modello di spiegare i dati.
- Il modello cattura efficacemente le relazioni presenti nel dataset.
- Dai coefficienti emerge una variabile predominante, mentre le altre hanno un impatto ridotto.

REGRESSIONE LASSO

- Con $\alpha = 1$, due coefficienti vengono azzerati, indicando la scarsa rilevanza delle relative variabili. Questo risultato è coerente con quanto osservato nella matrice di correlazione, che evidenziava forti dipendenze tra alcune caratteristiche.
- Si registra una lieve riduzione del **MSE** rispetto alla regressione lineare.

REGRESSIONE RIDGE

- La Ridge non annulla i coefficienti, ma ne riduce l'entità tramite regolarizzazione. Coefficienti molto vicini a zero suggeriscono la presenza di variabili poco informative.
- Il **MSE** risulta intermedio: migliore della regressione lineare, ma leggermente peggiore della Lasso.

FLASK

- Utilizzato la libreria Flask per creare un interfaccia web al programma
- La classe che sviluppa l'interfaccia permette all'utente di:
 - Caricare il dataset all'interno del programma e pulizia dei dati
 - Analisi dei dati con possibili grafici
 - Sviluppare un modello di machine learning per la predizione dei dati
 - Restituire delle valutazioni dei modelli creati

```
def __register_routes(self): 1 usage  ⌘ Federico Combi
    @self.app.route("/")  ⌘ Federico Combi
    def home():...

    @self.app.route("/info_csv")  ⌘ Federico Combi
    def info_csv():...

    @self.app.route("/plots")  ⌘ Federico Combi
    def plots():...

    @self.app.route("/outlier")  ⌘ Federico Combi
    def outlier():...

    @self.app.route(rule: "/regressione", methods=["POST"])  ⌘ Federico Combi
    def regressione():...
```

DOCKER

- Utilizzando *Dockerfile* creare un'immagine del nostro programma per poi eseguirla tramite Docker desktop in modo da avere la nostra applicazione in un container
- Questo processo permette di distribuire ed eseguire il programma in modo identico su qualsiasi ambiente, eliminando problemi di configurazione, dipendenze e compatibilità tra sistemi diversi.

```
FROM python:3.12
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "-m", "main"]
```

Contenuto Dockerfile